

# Habit Tracker Project Analysis & Optimization Report

## 1. Executive Summary

This document details the analysis and subsequent optimization of the Habit Tracker Django project. The initial analysis uncovered critical security vulnerabilities (IDOR) and significant performance bottlenecks (N+1 database queries). Furthermore, the frontend lacked dynamic visual engagement. This report outlines the steps taken to secure the backend, drastically improve database efficiency, and introduce a premium, motion-graphics-driven user interface.

## 2. Initial Analysis: Loopholes & Weak Points

### 2.1 Security Vulnerabilities (IDOR)

In the initial implementation, habit deletion (`mark_delete`) and modification (`toggle_today`, `edit_task`) did not verify user ownership. A malicious user could potentially manipulate or delete habits belonging to other users by simply altering the task primary key (PK) in the URL. This is known as Insecure Direct Object Reference (IDOR).

### 2.2 Performance Bottlenecks (N+1 Queries)

The 'home' dashboard view calculated daily progress by executing a database query for each day, for every single habit a user had. For a user with 10 habits over 2 weeks, this resulted in roughly 150+ individual database queries per page load. This highly inefficient loop structure would cause severe latency as the user base and habit count scaled.

## 3. Backend Optimization & Security Hardening

### 3.1 Securing Views

All modifying views were updated to enforce the `@login_required` decorator. Furthermore, database queries were modified to enforce strict ownership checks using Django's `get_object_or_404(task, pk=pk, user=request.user)`. Basic input validation was also added to habit creation to prevent empty submissions.

### 3.2 Algorithmic Query Optimization

The dashboard 'home' view was completely refactored. The nested database query loops were removed. Instead, the system now computes the required date range, queries all relevant 'TaskProgress' records in a single, optimized IN query, and constructs an in-memory hash map (dictionary). Progress lookups are now performed in  $O(1)$  time complexity against the dictionary. This reduced the database query count from  $O(N \times \text{Days})$  to exactly  $O(1)$ .

# Habit Tracker Project Analysis & Optimization Report

## 4. Frontend Motion Graphics & Premium UI

To provide a modern, premium user experience, the Animate On Scroll (AOS) library was integrated. This library leverages hardware-accelerated CSS transitions to animate DOM elements as they enter the viewport. Elements such as the hero section, statistic cards, and habit lists now feature smooth 'fade-up', 'zoom-in', and 'fade-left' entrance animations with staggered delays to guide the user's eye.

# Habit Tracker Project Analysis & Optimization Report

## 5. System Architecture & Flowchart

